

Topic 4 Assessment Documentation

Tracing, Logging and Debugging Agentic Loops

n8n Multi-Component Customer Assistant

Practical assessment guide and evidence plan based on the supplied assessment brief and workflow JSON.

1. Assessment Objective

The assessment evaluates the ability to inspect, debug, repair, and validate an AI Agent workflow by analyzing execution history, execution telemetry, reasoning loops, tool failures, and prompt guardrails. The objective is to diagnose workflow failures accurately, implement prompt-based error handling, optimize execution behavior, and validate successful workflow completion.

2. Assessment Exercises

Exercise	Required activity
1	Execute the broken workflow.
2	Inspect Execution History.
3	Analyze execution telemetry.
4	Identify configuration issues.
5	Repair the workflow using prompt guardrails.
6	Validate the repaired workflow.
7	Compare performance before and after repair.
8	Complete the end-to-end debugging mini project.

3. Supplied Workflow

The supplied workflow is named "Multi-Component Customer Assistant". Its main execution path is a Chat Trigger connected to the Customer Assistant Agent. The Agent is connected to an OpenAI Chat Model, Window Memory, calculator tool, and Structured Output Parser.

Workflow structure

```
Chat Trigger → Customer Assistant Agent
Customer Assistant Agent ← OpenAI Chat Model
Customer Assistant Agent ← Window Memory
Customer Assistant Agent ← calculator
Customer Assistant Agent ← Structured Output Parser
```

The uploaded JSON already contains a detailed error-handling and guardrail system prompt. Therefore, it should be treated as the repaired/target workflow configuration rather than automatically assuming it is the original broken version.

4. Agent Configuration

Setting	Supplied configuration
Agent	Customer Assistant Agent

Setting	Supplied configuration
Model	OpenAI Chat Model using gpt-4.1
Memory	Window Memory; context window length 10
Tool	calculator
Output parser	Structured Output Parser
Max iterations	3
Temperature	0
Max tokens	1000

5. Key Prompt Guardrails in the Repaired Workflow

Grounding: Only facts provided by the customer or returned by a tool may be used. The Agent must not invent names, order numbers, products, prices, quantities, discounts, tax rates, or other details.

Missing information: If a required detail is missing, the Agent must ask a brief clarifying question instead of fabricating a value. It must set `calculation_used` to false and result to an empty string.

Arithmetic: For any arithmetic, the calculator tool must be called with the exact expression and its returned value must be used. The Agent must not calculate or estimate the answer itself.

Duplicate calculations: The calculator should be called only as many times as needed, and an identical calculation already performed in the same turn must not be repeated.

Tool failure: If the calculator fails, is unavailable, or returns an error, the Agent must not silently retry repeatedly and must not guess the numeric answer.

Graceful failure: On calculator failure, the Agent should stop, apologize briefly, explain that the calculation could not be completed, set `calculation_used` to false, and set result to an empty string.

Honesty: If the question cannot be answered with the available information and tools, the Agent should say so honestly rather than fabricate a response.

6. Required Structured Output

The workflow uses a Structured Output Parser with the following intended fields:

```
{"answer": "natural-language reply", "calculation_used": true/false, "result": "numeric result or empty string"}
```

The supplied schema requires `answer` and `calculation_used`. The `result` field is defined as a string and is intended to contain the numeric result when a calculation is performed, otherwise an empty string.

7. Practical Debugging Procedure

1. Execute the broken workflow and capture the initial failure.
2. Open n8n Execution History and select the failed execution.
3. Inspect the AI Agent execution and identify the tool call sequence.
4. Inspect execution telemetry, including execution timing and repeated tool behavior.
5. Identify the configuration or prompt issue responsible for poor failure handling.
6. Update the Agent System Message with explicit error-handling guardrails.
7. Re-execute the workflow using the same or equivalent test scenario.
8. Verify that a calculator failure is handled gracefully and does not trigger repeated identical tool execution.
9. Capture the successful repaired execution as evidence.
10. Compare the before and after behavior and document the improvement.

8. Before vs. After Evidence

Evidence area	Before repair	After repair
Tool failure	Capture the actual failed tool event.	Show that the failure is handled rather than hidden.
Repeated tool execution	Capture repeated/undesired calls if present.	Show that repeated identical retry behavior is prevented.
Agent response	Show the unsuccessful or poorly handled response.	Show a concise, honest graceful error response.
<code>calculation_used</code>	Record the actual value observed.	false when the calculator was not successfully used.

Evidence area	Before repair	After repair
result	Record the actual value observed.	Empty string when calculation cannot be completed.
Execution timing	Record telemetry from the failed execution.	Record telemetry from the repaired execution.

9. Screenshot / Evidence Checklist

- Broken workflow canvas
- Broken execution result
- Execution History entry
- AI Agent execution details
- HTTP/tool failure evidence, if present in the supplied/broken scenario
- Execution timing / telemetry
- Relevant Agent configuration
- Original System Prompt or pre-repair state
- Updated System Prompt with guardrails
- Repaired workflow canvas
- Successful repaired execution
- Evidence that repeated tool retry was prevented
- Before/after performance comparison

10. Final Deliverables

Deliverable	Status in assessment brief
Loom/YouTube Demo Video	Mandatory
Public GitHub Repository	Mandatory
Exported n8n Workflow JSON	Mandatory
Documentation PDF/DOCX	Recommended
Workflow Screenshots	Mandatory
Execution Evidence — Before & After	Mandatory
README	Mandatory

11. Demo Video Flow

1. Introduce the Topic 4 assessment and the Multi-Component Customer Assistant workflow.
2. Show the broken workflow and execute it.
3. Open Execution History and identify the failure.
4. Inspect the AI Agent and relevant tool execution/telemetry.
5. Explain the root cause: insufficient error-handling behavior or configuration in the broken state.
6. Show the updated System Prompt guardrails.
7. Execute the repaired workflow.
8. Show graceful handling and the absence of repeated tool retry.
9. Show the before/after evidence and summarize the improvement.

12. GitHub Repository Suggested Structure

```
topic-4-agentic-loop-debugging/  
■■■ workflow/  
■ ■■■ Multi-Component-Customer-Assistant.json  
■■■ screenshots/  
■ ■■■ before/  
■ ■■■ after/  
■■■ evidence/  
■ ■■■ before-after-comparison.md  
■■■ documentation/  
■ ■■■ Topic-4-Debugging-Documentation.pdf  
■■■ README.md
```

13. Important Source Note

This document preserves the terminology and requirements from the supplied Topic 4 assessment brief. The workflow-specific configuration is based on the uploaded “Multi-Component Customer Assistant” JSON. The uploaded JSON already includes the repaired prompt guardrails, so actual before-state evidence must come from the original broken workflow execution rather than being invented from the repaired JSON.